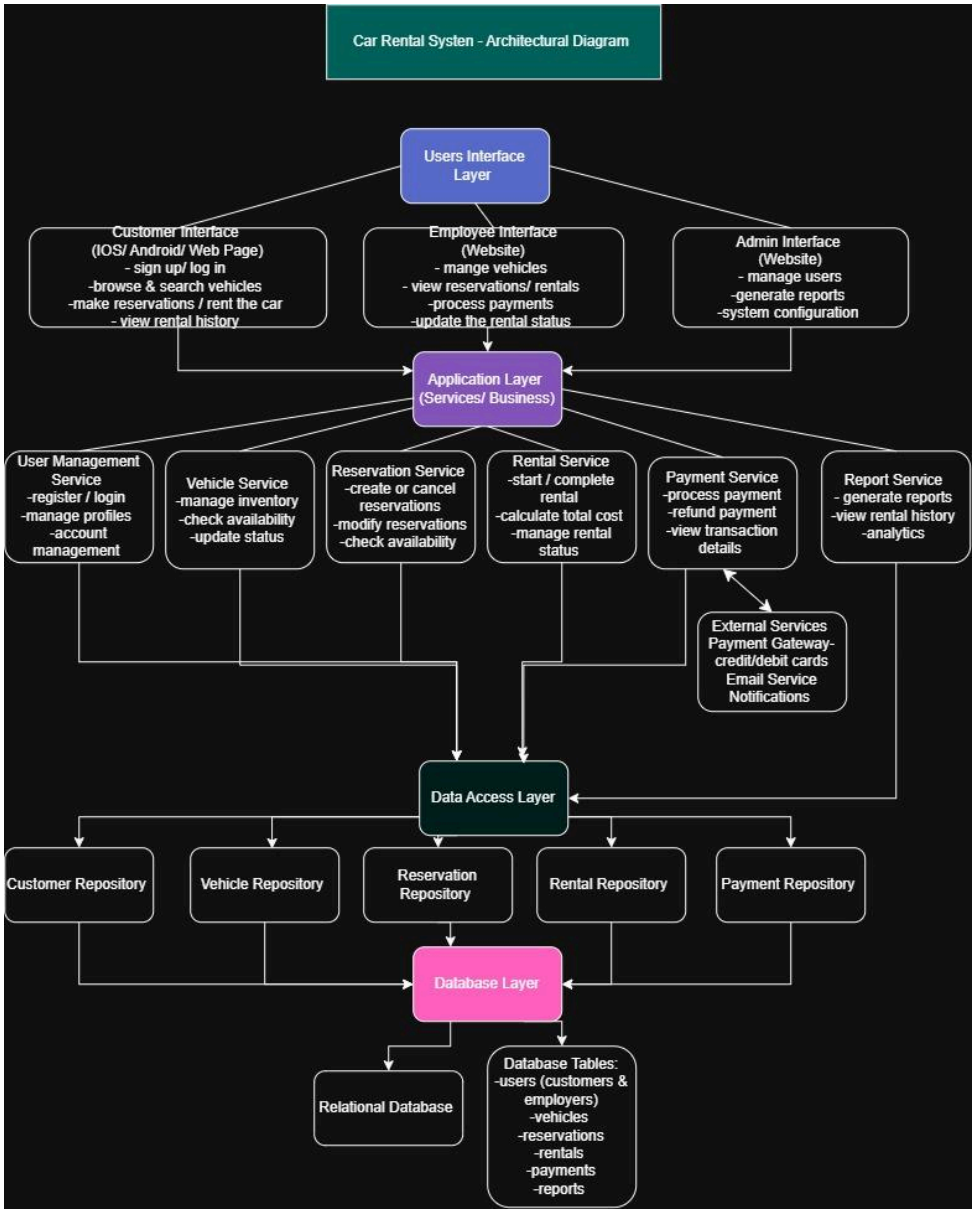
BeAvis Car Rental System

Chris E, Kevin P, Tigist T, Michelle V, Light O

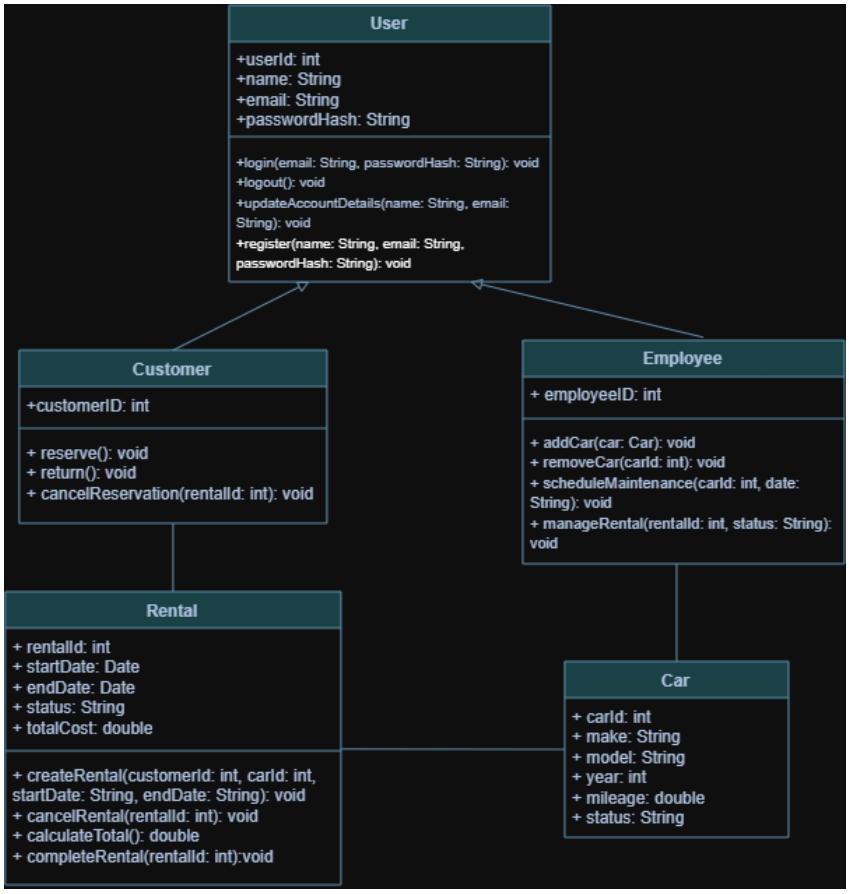
This is a car rental system designed to manage a car rental company. This system will allow customers to rent vehicles both online and via a mobile app, provided they have signed up for an account with the company. This system will also facilitate the distribution and signing of rental agreement contracts under secure conditions.

Employees will also be able to access the system in order to look up customers. Upon looking up a customer in the system, they will be able to see the rental status and contracts, check what cars are currently available, and update the status of cars.

Architectural Diagram:



UML Diagram:



User class

Description

The User class represents a general person who uses the BeAvis system. It is the parent class for the Employee and Customer classes. This class stores common account information and allows users to create accounts, log in, log out, and update their personal details. Passwords should not be stored as plain text. Instead, the system should store a secure password hash.

Attributes

- userId: int
- name: String
- email: String
- passwordHash: String

Operations

- register(name: String, email: String, password: String): boolean
- login(email: String, password: String): boolean
- logout(): void
- updateAccountDetails(name: String, email: String): void

Employee Class

Description

The Employee class represents a BeAvis employee; it inherits the common account information and operations from the User class. Employees have additional permissions because they manage vehicles, customers, rentals, and maintenance information. Employees can update the availability of cars and review customers' rental information

Attributes

- employeeId: int

Operations

+addCar(car: Car): void
+removeCar(carId: int): boolean
+scheduleMaintenance(carId: int, description: String): void
+manageRental(rentalId: int, action: String): boolean

Rental Class

Description

The rental class represents a vehicle reservation made by a customer. It stores information about the rental dates, rental status, and total price. A rental is connected to one customer and one car. The class allows the system to create, cancel, calculate, and complete rental transactions.

Attributes

- rentalId: int
- startDate: Date
- endDate: Date
- status: String
- totalCost: double

Operations

+ createRental(customerId: int, carId: int,
startDate: Date, endDate: Date): Rental
+ cancelRental(): boolean
+ calculateTotal(dailyRate: double): double
+ completeRental(): void

Car Class

Description

The car class represents a vehicle in the BeAvis inventory; it stores information about the car, including model, year, mileage, and current status. The car status shows whether the vehicle is available for reservation, rented, or under maintenance

Attributes

- carId: int
- make: String
- model: String
- year: int
- mileage: double
- status: String

Operations

+ updateStatus(newStatus: String): void
+ getDetails(): String
+ isAvailable(startDate: Date, endDate: Date): boolean

Title page: Christopher
Software title: Christopher
Team members: Christopher
System Description: Christopher
Brief overview of system: Christopher
Software Architecture Overview
Architectural diagram of all major components: Michelle
UML Class Diagram: Kevin
Descriptions: Tigist
Development plan and timeline: Light
Partitioning of tasks: Light
Team member responsibilities: Light

The Software Design Specification (Architecture Overview and UML Class Diagram) remains unchanged from Assignment 2, as the current design fully supports all required attributes and operations for unit, integration, and system verification testing.

Verification Test Plan: BeAvis Car Rental System

- ❖ Feature 1: Car Availability Tracking & Search

1. Unit Test

```
Car c
c.status = "AVAILABLE"

if (c.status == "AVAILABLE")
   return PASS
else
   return FAIL
```

Explanation: This unit test ensures that a Car object correctly holds and exposes its internal status attribute in memory.

2. Integration Test

```
Car c
c.carId = 42
c.status = "AVAILABLE"

Customer cust
cust.reserve()

if (c.status == "RESERVED")
   return PASS
else
   return FAIL
```

Explanation: This integration test verifies the interaction when a Customer executes the reserve() method. It ensures that placing a reservation successfully changes the linked Car object's status from "AVAILABLE" to "RESERVED" across the domain models.

3. System Test

Description: A customer logs into the BeAvis app and navigates to the "browse & search vehicles" page. They browse available vehicles in the inventory. Car 42 is currently flagged with status = "RESERVED" in the database, while Car 43 has status = "AVAILABLE". The system filters the inventory view so that Car 42 is hidden from the customer's browse list, while Car 43 is displayed as available for selection.

Explanation: This end-to-end system test verifies the vehicle search workflow across all layers. It ensures that the Customer Interface, Vehicle Service, Vehicle Repository, and Relational Database work together to restrict unavailable cars from appearing in customer search views.

❖ Feature 2: Rental Cancellation & Inventory Release

1. Unit Test

```
Rental r
r.rentalId = 101
r.status = "RESERVED"

r.cancelRental(101)

if (r.status == "CANCELLED")
   return PASS
else
   return FAIL
```

Explanation: This unit test checks the cancelRental(rentalId: int) method in the Rental class. It verifies that passing the target rentalId updates the object's status attribute from "RESERVED" to "CANCELLED".

2. Integration Test

```
Rental r
r.rentalId = 101
r.carId = 42

Car c
c.carId = 42
c.status = "RESERVED"

Customer cust
cust.cancelReservation(101)

if (c.status == "AVAILABLE")
   return PASS
else
   return FAIL
```

Explanation: This integration test checks how Customer.cancelReservation(rentalId: int) interacts with Rental and Car objects. It verifies that when a customer cancels a reservation, the system updates the linked Car object's status back to "AVAILABLE" so it can be booked again.

3. System Test

Description: A customer logs into their account on the BeAvis app and views their active rental history. They select rental #101 and click "Cancel Reservation" (triggering cancelReservation(101)). The system processes the request, updates the rental record to 'CANCELLED' in the rentals database table, restores car #42 status back to 'AVAILABLE' in the vehicles database table, and updates the screen with a cancellation success message.

Explanation: This system test verifies the cancellation flow across all system layers from UI user input, down to application services, repository updates, and database persistence.